Toward Automated Discovery of Asymmetric Mempool DoS in Blockchains

Yibo Wang, Yuzhe Tang, Kai Li, and Wanning Ding

***Abstract*—In blockchains, mempool controls transaction flow before consensus, denial of whose service hurts the health and security of blockchain networks. This paper presents MPFUZZ, the first mempool fuzzer to find asymmetric DoS bugs by exploring the space of symbolized mempool states and optimistically estimating the promisingness of an intermediate state in reaching bug oracles. Compared to the baseline blockchain fuzzers, MPFUZZ achieves a $>100\times$ speedup in finding known DETER exploits. Running MPFUZZ on major Ethereum clients leads to discovering new mempool vulnerabilities, which exhibit a wide variety of sophisticated patterns, including stealthy mempool eviction and mempool locking. Rule-based mitigation schemes are proposed against all newly discovered vulnerabilities.**

***Index Terms*—Stateful fuzzing, asymmetric DoS, blockchain mempool, economics of security, symbolized fuzzing.**

I. Introduction

In Ethereum, a mempool buffers unconfirmed transactions from web3 users before they are included in the next blocks. Mempool provides the essential functionality to bridge the gap between varying rates of submitted transactions and produced blocks. As shown in recent studies [1], denying a mempool service can force the blockchain to produce blocks of low or even zero (Gas) utilization, undermining validators' incentives and shrinking the blockchain networks in the long run, re-introducing the 51% attacks. Besides, a denied mempool service can prevent normal transactions from block inclusion, cutting millions of web3 users off the blockchain and failing the decentralized applications relying on real-time blockchain access.

Problem: Spamming the mempool to deny its service has been studied for long [2], [3], [4], [5]. Early designs by sending spam transactions at high prices burden attackers with high costs and are of limited practicality. What poses a real threat is Asymmetric DeniAl of Mempool Service, coined by ADAMS, in which the mempool service is denied at an asymmetrically low cost. That is, the attack costs, in terms of the fees of adversarial transactions, are significantly lower than those of normal transactions victimized by the denied mempool. In the existing literature, DETER [1] is the first ADAMS attack, and it works by sending invalid transactions to *directly evict* normal transactions in the mempool. MemPurge [6] is a similar mempool attack that finds a way to send overdraft transactions into Geth's pending transaction pool and causes eviction there. These known attacks are easy to detect (i.e., following the same direct-eviction pattern). In fact, the DETER bugs reported in 2021 have been successfully fixed in all major Ethereum clients as of Fall 2023, including Geth, Nethermind, Erigon, and Besu. Given this state of affairs, we pose the following research question: Are there new ADAMS vulnerabilities in the latest Ethereum clients already patched against direct-eviction based attacks?

This work takes a systematic and semi-automated approach to discovering ADAMS vulnerabilities, unlike the existing DETER bugs that are manually found. Fuzzing mempool implementations is a promising approach but also poses unique challenges: Unlike the consensus implementation that reads only valid confirmed transactions, the mempool, which resides in the pre-consensus phase, needs to handle various unconfirmed transactions, imposing a much larger input space for the fuzzer. For instance, a mempool can receive *invalid transactions under legitimate causes*,[1] and factors such as fees or prices are key in determining transaction admission outcomes. Existing blockchain fuzzers including Fluffy [7], Loki [8] and Tyr [9] all focus on fuzzing consensus implementation and don't explore the extra transaction space required by mempool fuzzing. As a result, directly re-purposing a consensus fuzzer to fuzz mempool would be unable to detect the DETER bugs, let alone discover more sophisticated new ADAMS bugs.

Proposed methods: To efficiently fuzz mempools, our key observation is that real-world mempool implementation admits transactions based on abstract "symbols", such as pending and future transactions, instead of concrete value. Thus, *sending multiple transactions under one symbol would trigger the same*

Received 27 October 2025; revised 26 March 2026 and 19 May 2026; accepted 28 May 2026. Date of publication 1 June 2026; date of current version 14 August 2026. The work of Yibo Wang was supported by the Ethereum Foundation's 2025 Academic Grants. The work of Wanning Ding and Yuzhe Tang were supported in part by the two Ethereum Foundation awards, NSF under Grant CNS-2139801, Grant CNS-1815814, and Grant DGE2104532; and in part by the Flashbots Research Proposal award. The work of Kai Li was supported by the NSF under Grant CNS-2602850. Recommended for acceptance by T. Yue. *(Corresponding authors: Yuzhe Tang; Yibo Wang.)*

Yibo Wang is with the University of Wyoming, Syracuse, NY 13244 USA (e-mail: ywang34@uwyo.edu).

Yuzhe Tang and Wanning Ding are with the Syracuse University, Laramie, WY 82071 USA (e-mail: ytang100@syr.edu; wding04@syr.edu).

Kai Li is with Stevens Institute of Technology, Hoboken, NJ 07030 USA (e-mail: kli50@stevens.edu).

Digital Object Identifier 10.1109/TSE.2026.3698791

[1]For instance, future transactions can be caused by out-of-order information propagation in Ethereum.

mempool behavior repeatedly, and it suffices to explore just one transaction per symbol during fuzzing without losing the diversity of mempool behavior.

We propose symbolized-stateful mempool fuzzing or MPFUZZ. To begin with, MPFUZZ is set up and run in a three-step workflow: It is first manually set up against a size-reduced mempool under test, which induces a much smaller search space for fuzzing. Second, MPFUZZ is iteratively run against the reduced mempool to discover short exploits. Third, MPFUZZ automatically extends the short exploits to actual ones that are functional on real mempools and Ethereum clients. Internally, the design of MPFUZZ is based on seven transaction symbols we design out of Ethereum semantics: normal, future, parent, child, overdraft, latent overdraft, and replacement transactions. Under these symbols, in each iteration of fuzzing, MPFUZZ explores one concrete transaction per symbol, sends the generated transaction sequence to the target mempool, observes the mempool end state, and extracts the feedback of symbolized state coverage and state promisingness in reaching bug oracles to guide the next round of fuzzing. In particular, MPFUZZ employs a novel technique to evaluate state promisingness, that is, by *optimistically* estimating the costs of unconfirmed transactions whose validity is subject to change in the future. After finding a short exploit against the reduced mempool, MPFUZZ automatically extends the small exploit to a more extensive exploit that is effective within a larger mempool. Specifically, MPFUZZ extracts all the unique admission event patterns in the small exploit and aligns these patterns to construct the attack transaction sequence on the larger-size mempool.

Found attacks: We run MPFUZZ on six leading execution-layer Ethereum clients deployed on the mainnet's public-transaction path (Geth [10], Nethermind [11], Erigon [12], Besu [13], Reth [14], and OpenEthereum [15]), three PBS builders (proposer-builder separation) on the mainnet's private-transaction and bundle path (Flashbot $v1.11.5$ [16], EigenPhi [17] and bloXroute [18]), and three Ethereum-like clients (BSC $v1.3.8$ [19] deployed on Binance Smart Chain, go-opera $v1.1.3$ [20] on Fantom, and core-geth $v1.12.19$ [21] on Ethereum Classic). On Ethereum clients of historical versions, MPFUZZ can rediscover known DETER bugs. On the latest Ethereum clients where DETER bugs are fixed, MPFUZZ can find new ADAMS attacks, described next.

We summarize the newly discovered ADAMS exploits into several patterns: 1) Indirect eviction by valid-turned-invalid transactions. Unlike the direct-eviction pattern used in DETER, the indirect-eviction attack works more stealthily in two steps: The attacker first sends normal-looking transactions to evict victim transactions from the mempool and then sends another set of transactions to *turn* the admitted normal-looking transactions into invalid ones, bringing down the cost. 2) Locking mempool, which is to occupy the mempool to decline subsequent victim transactions. Mempool locking does not need to evict existing transactions from the mempool as the eviction-based DETER attacks do. 3) Adaptive attack strategies where the attack composes adversarial transactions in an adaptive way to the specific policies and implementations of the mempool under test. Example strategies include composing transactions of multiple patterns in one attack, re-sending evicted transactions to evict "reversible" mempools, locking mempool patched against turning, etc.

Systematic evaluation: We conducted an ablation study to evaluate the performance of MPFUZZ, which incorporates three distinct techniques. We aim to identify the contribution and significance of each component to the overall MPFUZZ. We implement four baseline fuzzers to compare against MPFUZZ. The first baseline is a stateless fuzzer built within the GoFuzz framework [22]. For the remaining three baselines, each one selectively disables a single technique from MPFUZZ: symbolized state, symbolized input, and promising feedback and energy. The experimental result shows that the stateless fuzzer is the least effective in discovering ADAMS exploits. Among the three distinct techniques, the symbolized input technique improves efficiency more compared to that of the promising feedback and symbolized state coverage, and the state promisingness in feedback is more beneficial for performance than symbolized state coverage.

This paper extends our prior conference paper [23]. Compared with the conference version, this journal version makes several substantial additions. First, it adds an automated exploit extension mechanism that extends short exploits found on reduced mempools to larger mempools. Second, it strengthens the methodology by further formalizing the state promisingness metric and the exploit extension process. Third, it expands the motivation by clarifying why mempool fuzzing is more difficult than fuzzing other blockchain components. Fourth, it broadens the related-work discussion by adding more concrete comparisons with existing fuzzing approaches. Fifth, it adds new evaluations, including fuzzing performance, runtime cost, and the effectiveness of the exploit extension algorithm. Finally, it expands the discussion of generalization to other blockchain systems, the rationale and limitations of transaction symbolization, and the fee model assumptions. These additions substantially extend both the methodology and the evaluation beyond the conference version.

Contributions: The paper makes contributions as follows:

- *New fuzzing problem*: This paper is the first to formulate the mempool-fuzzing problem to automatically find asymmetric DoS vulnerabilities as bug oracles. Fuzzing mempools poses new unique challenges that existing blockchain fuzzers don't address and entails a larger search space including invalid transactions and varying prices.
- *New fuzzing method*: The paper presents the design and implementation of MPFUZZ, a symbolized-stateful mempool fuzzer. Given a mempool implementation, MPFUZZ defines the search space by the mempool states covered under symbolized transaction sequences and efficiently searches this space using the feedback of symbolized state coverage and the promising-ness of an intermediate state in triggering bug oracles. With a Python prototype and runs on real Ethereum clients, MPFUZZ achieves a $> 100\times$ speedup in finding known DETER exploits compared to baselines.
- *New discovery of mempool vulnerabilities*: MPFUZZ discovers new asymmetric-DoS vulnerabilities in six major Ethereum clients, PBS and Ethereum-like clients. We

found and reported 24 newly discovered ADAMS bugs. After our reporting, 15 bugs are confirmed and 3 bugs are fixed.

- *New evaluation of fuzzing performance*: The paper presents a systematic evaluation of MPFUZZ's fuzzing performance, including ablation study, fuzzing efficiency, runtime cost, and exploit-extension effectiveness. The results show that MPFUZZ efficiently explores deep mempool states and discovers exploits more effectively than alternative designs.

II. Related Works

Generic fuzzers: Many fuzzers improve AFL-style coverage-guided fuzzing by enhancing input mutation, seed scheduling, or feedback design. Alphuzz [24] improves seed scheduling with Monte Carlo Tree Search (MCTS). Its key idea is that the mutation relation among seeds can be organized as a seed-mutation tree, and the next seed is selected by balancing exploitation and exploration on that tree. Deep Reinforcement Fuzzing [25] formulates fuzzing as a reinforcement learning problem and learns mutation policies from runtime rewards. SyzVegas [26] applies an adversarial multi-armed bandit algorithm to optimize task and seed scheduling in kernel fuzzing. IJON [27] augments AFL-style fuzzing with lightweight user annotations to expose deep program states that pure automation may fail to reach. These works improve generic fuzzing efficiency, but they mainly assume that the input space can be explored through direct mutation over raw bytes, syscalls, or branch-related values. In contrast, mempool fuzzing must reason about transaction semantics and evolving admission states over transaction sequences. As a result, directly applying these generic strategies is insufficient for effectively exploring mempool-specific attack behaviors.

Consensus fuzzers: Fluffy [7] is a code-coverage based differential fuzzer to find consensus bugs in Ethereum Virtual Machines (EVM). Specifically, given an EVM input, that is, a transaction sequence, Fluffy sends it to multiple nodes running different Ethereum clients (e.g., Geth and OpenEthereum) for execution. 1) The test oracle in Fluffy is whether the EVM end states across these nodes are different, implying consensus failure. 2) Fluffy mutates ordered transaction sequences at two levels: it reorders/adds/deletes transactions in the sequence, and for each new transaction to generate, it randomly selects values in Gas limits and Ether amounts. For the data field, it employs semantic-aware strategies to add/delete/mutate bytecode instructions of pre-fixed smart contract templates. 3) Fluffy uses code coverage as feedback to guide the mutation.

Loki [8] is a stateful fuzzer to find consensus bugs causing crashes in blockchain network stacks. Specifically, Loki runs as a fuzzer node interacting with a tested blockchain node through sending and receiving network messages. 1) The test oracle in Loki is whether the tested blockchain node crashes. 2) The fuzzer node uses a learned model of blockchain network protocol to generate the next message of complying format, in which the content is mutated by randomly choosing integers and bit-flipping strings. In other words, the mutation in Loki is unaware of application level semantics. For instance, if the message is about propagating an Ethereum transaction, the transaction's nonce, Ether amount, and GasPrice are all randomly chosen; the sender, receiver, and data are bit flipped. 3) Loki records messages sent and received from the test node. It uses them as the state. New states receive positive feedback.

Tyr [9] is a property-based stateful fuzzer that finds consensus bugs causing the violation of safety, liveness, integrity, and other properties in blockchain network stacks. Specifically, in Tyr, a fuzzer node is connected to and executes a consensus protocol with multiple neighbor blockchain nodes. 1) The test oracle Tyr uses is the violation of consensus properties, including safety (e.g., invalid transactions cannot be confirmed), liveness (e.g., all valid transactions will be confirmed), integrity, etc. The checking is realized by matching an observed end state with a "correct" state defined and run by the Tyr fuzzer. 2) Tyr mutates transactions on generic attributes like randomly selecting senders and varying the amount of cryptocurrencies transferred (specifically, with two values, the full sender balance, and balance plus one). When applied to Ethereum, Tyr does not mutate Ethereum-specific attributes, including nonce or GasPrice. It generates one type of invalid transaction, that is, the double-spending one, which in account-based blockchains like Ethereum are replacement transactions of the same nonce. 3) Tyr uses state divergence as the feedback, that is, the difference of end states on neighbor nodes receiving consensus messages in the same iteration. In Tyr, the states on a node include both the messages the node sent and received, as well as blocks, confirmed transactions, and other state information.

Fuzzing for distributed systems: Prior work has also applied fuzzing to other distributed systems with domain-specific abstractions. Chronos [28] targets timeout bugs in distributed systems by injecting transient delays into runtime libraries and mutating delay-activation sequences under deep-priority guidance. Monarch [29] fuzzes distributed file systems through a multi-node architecture and a two-step mutator that combines syscall mutation with distributed fault injection. AMPFUZZ [30] discovers amplification DDoS vulnerabilities in UDP services through protocol-aware greybox fuzzing.

These works show the importance of domain-specific modeling when fuzzing complex distributed software. However, their mutation spaces and target behaviors are different from those in blockchain mempools. Chronos mutates delay patterns under a fixed workload, Monarch mutates syscall and fault sequences, and AmpFuzz focuses on amplification behavior in UDP services. None of them model the semantics of unconfirmed blockchain transactions or the admission and eviction policies that govern pre-consensus transaction handling.

Position of MPFUZZ: MPFUZZ differs from prior work in four main aspects. First, it targets asymmetric denial-of-mempool-service vulnerabilities rather than crashes, consensus failures, timeout bugs, or execution inconsistencies. Second, it introduces transaction symbolization to reduce the mempool input space into symbol space. Third, it uses symbolized state coverage and state promisingness to guide exploration toward deep mempool states. Fourth, it models mempool behavior as a stateful admission problem over transaction sequences, addressing a distinct pre-consensus attack surface that is not covered by existing fuzzers.

III. Background and Challenges

Ethereum mempools: In Ethereum, users send their transactions to the blockchain network, which are propagated to reach validator nodes. On every blockchain node, unconfirmed transactions are buffered in the mempool before they are included in the next block or evicted by another transaction.

The core mempool design is transaction admission: Given an initial state, whether and how the mempool admits an arriving transaction. In practice, example policies include those favoring admitting transactions of higher prices (and thus evicting or declining the ones of lower prices), transactions arriving earlier, certain transaction types (e.g., parent over child transactions), and valid transactions (over future transactions). See different types of Ethereum transactions in § 4-A.

Challenges of mempool fuzzing: Fuzzing mempool implementations is fundamentally different from fuzzing other blockchain components such as consensus fuzzing. A key difference is that consensus-layer fuzzing mainly explores valid transactions, whereas mempool fuzzing targets the pre-consensus phase, where the system must handle a much broader set of unconfirmed transactions, including both valid and legitimately invalid ones. As a result, mempool fuzzing faces a substantially larger input space.

Another key difference is transaction ordering. Given a transaction set, consensus-layer execution typically follows a fixed execution order. In contrast, mempool fuzzing must also consider different transaction arrival orders, because in a real mempool, transactions may arrive in different sequences, and the arrival order can directly affect admission decisions and the resulting mempool state. In addition, each transaction field, such as nonce, GasPrice, Ether amount, and sender, has a large value range, which further enlarges the search space.

A concrete quantitative example can be given by considering only one transaction field, namely the nonce. Consider an account whose current confirmed nonce is 3, and assume the largest nonce explored is 1000. For consensus fuzzing, it is sufficient to test the next executable transaction with nonce 4. In contrast, mempool fuzzing must also consider higher nonces, namely 5,6, . . ., 1000, because such transactions may be admitted as future transactions and later affect admission and eviction behavior. Thus, even when considering only the nonce field, consensus fuzzing may need to explore one transaction, while mempool fuzzing may need to explore 997 candidate nonces in total, including 996 additional future transactions. This gap becomes even larger when combined with other transaction fields such as GasPrice, Ether amount, and sender.

Existing blockchain fuzzers do not efficiently explore this additional transaction space. Consider again an account whose current confirmed nonce is 3. To construct a future transaction, the fuzzer must first know that nonce 4 is the next executable one, and then deliberately generate a transaction with a higher nonce, such as 5, so that it will be treated as a future transaction. This is not a mutation problem over one independent transaction. It requires the fuzzer to model the sender's current pending sequence and the nonce gap. Existing fuzzers do not model these Ethereum-specific admission conditions. Fluffy focuses on EVM execution after admission, where it is sufficient to execute the next valid transaction with nonce 4. Loki randomly chooses integers for transaction nonce without application-level transaction semantics, so although it may occasionally generate nonce 5, it does not do so in a guided way that depends on first recognizing nonce 4 as the next executable one. Tyr does not mutate Ethereum-specific attributes such as nonce. In this example, it would use the next executable nonce 4 rather than deliberately constructing the future transaction with nonce 5. As a result, these fuzzers do not systematically construct the nonce or other transaction fields needed to reach future or other exploit-relevant invalid transaction states.

TABLE I
Notations

Symbol	Meaning	Symbol	Meaning
m	Mempool length	ops	Tx sequence
st	State of residential txs in mempool	dc	History of declined txs from mempool
A	Adversarial tx sender	B	Benign tx sender
$tx\left[\begin{smallmatrix} s & v \\ n & f \end{smallmatrix}\right]$	A transaction from sender s, of nonce n, transferring Ether value v, and with GasPrice f		

IV. Threat Model and Bug Oracle

In the threat model, an attacker controls one or few nodes to join an Ethereum network, discovers and neighbors critical nodes (e.g., top validators, backends of an RPC Service, MEV searcher) if necessary, and sends crafted adversarial transactions to their neighbors. The attacker in this work has the same networking capacity as in DETER [1].

The attacker's goal is to disable the mempools on the critical nodes or all nodes in an Ethereum network and cause damage such as deployed or dropped transactions in block inclusion, produced blocks of low or zero (Gas) utilization, and disabled other downstream services. In practice, this damage is beneficial to competing block validators, MEV searchers, and RPC services. Besides, the attacker aims to cause this damage at asymmetrically low costs, as will be described.

A. Notations

Transactions: In Ethereum, we consider two types of accounts: a benign account of index i denoted by B_i, and an adversarial account of index i denoted by A_i. A (concrete) Ethereum transaction tx is denoted by $tx\left[\begin{smallmatrix} s & v \\ n & f \end{smallmatrix}\right]$, where the sender is s, the nonce is n, the GasPrice is f, and the transferred value in Ether is v. This work does not consider the "data" field in Ethereum transactions or smart contracts because smart contracts do not affect the validity of the Ethereum transaction. The notations are summarized in Table I.

States and transitions: We recognize two mempool-related states in Ethereum, the collection of transactions stored in mempool st, and the collection of declined transactions dc.

Suppose a mempool of state $\langle st_i, dc_i \rangle$ receives an arriving transaction tx_i and transitions to state $\langle st_{i+1}, dc_{i+1} \rangle$.

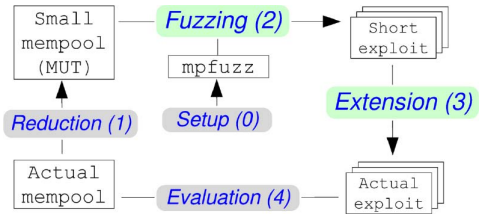

Fig. 1. Overview of exploit discovery and evaluation workflow: 0) MPFUZZ setup, 1) mempool reduction, 2) fuzzing on reduced mempool under test (MUT) to discover short exploits, and 3) exploit extension. The extended exploits are 4) evaluated on the actual mempool of the original size. Green means automated tasks, and gray requires manual effort.

An arriving transaction can be admitted with eviction, admitted without eviction, or declined by a mempool. The three operations are defined as follows. 1) An arriving transaction tx_i is admitted with evicting a transaction tx'_i from the mempool, if $st_{i+1} = st_i \setminus tx'_i \cup tx_i, dc_{i+1} = dc_i$. 2) tx_i is admitted without evicting any transaction in the mempool, if $st_{i+1} = st_i \cup tx_i, dc_{i+1} = dc_i$. 3) tx_i is declined and does not enter the mempool, if $st_{i+1} = st_i, dc_{i+1} = dc_i \cup tx_i$.

B. Exploit Discovery Workflow

To set up the stage, we present the workflow overview in this work. The workflow is depicted in Fig. 1. To discover vulnerabilities and exploits in an actual mempool, one has to 0) set up MPFUZZ by various parameters (as will be introduced), 1) reduce the mempool from its original configuration to a much smaller version, named mempool under test (MUT). 2) MPFUZZ is run on the reduced MUT deployed locally (with settings described in § 7-A). Fuzzing leads to the discovery of short exploits. 3) Short exploits are extended to the longer ones, that is, actual exploits. 4) The success of the actual exploit is evaluated on the actual mempools deployed in close-to-operational settings, such as using popular Ethereum testnets like Goerli or a network we set up locally running unmodified Ethereum clients. Fuzzing a reduced mempool (MUT) instead of an original one is necessary to ensure the efficient execution of the MPFUZZ, which entails many iterations of re-initiating and populating the mempool.

In this workflow, fully automated is Step 2) and Step 3), that is, the discovery of short exploits on a given MUT and extension of the short exploit to an actual exploit (colored green in Fig. 1). Other steps colored in gray require manual efforts and are described below: In **Step 0)**, MPFUZZ setup entails setting parameters in bug oracles (e.g., ϵ and λ) and fixed symbols in guiding fuzzing. The setup is usually one-time. In **Step 1)**, mempool reduction entails reconfiguring the mempool to a much smaller capacity (e.g., MUT length $m = 6$, compared to the original size like $m' = 6144$ as in Geth). In addition, policy-specific parameters in the actual mempool are tuned down proportionally. For instance, the Geth mempool buffers up to $py'_1 = 1024$ future transactions, and when buffering more than $py'_3 = 5120$ pending transactions, the mempool triggers a protective policy to limit the number of transactions sent from the same sender by $py'_2 = 16$. In a reduced MUT of $m = 6$, these parameters are reduced while retaining the same proportion, such as $py_1 = 1, py_3 = 5, py_2 = 2$. The reduced MUT is constructed by adjusting only the built-in capacity-related configuration of the mempool, without changing the underlying implementation of transaction handling. Therefore, the reduced MUT preserves the same core mempool semantics as the original one, including transaction validation, admission decisions, replacement handling, nonce ordering, and eviction logic. Thus, mempool reduction changes the size of the state space, but not the decision logic exercised by MPFUZZ. Thus, the reduced MUT is intended to preserve the mempool semantics relevant to exploit discovery while making the search space tractable for fuzzing.

C. Bug Oracle

Definition 4.1 (Tx admission timeline): In a transaction admission timeline, a mempool under test is initialized at state $\langle st_0, dc_0 = \varnothing \rangle$, receives a sequence of arriving transactions ops, and ends up with state $\langle st_n, dc_n \rangle$. Then, a validator continually builds blocks by selecting and clearing transactions in the mempool until the mempool is empty. This transaction timeline is denoted by $\langle st_0, dc_0 = \varnothing \rangle, ops \Rightarrow \langle st_n, dc_n \rangle$.

The transaction admission timeline is simplified from the timeline that an operational mempool experiences where transaction arrival and block building can be interleaved.

We use two mempool-attack templates to define two bug oracles: an eviction-based DoS where adversarial transactions evict existing victim transactions in the mempool, and a locking-based DoS where existing adversarial transactions in the mempool decline arriving victim transactions.

Definition 4.2 (Eviction bug oracle): A transaction admission timeline, $\langle st_0, dc_0 = \varnothing \rangle, ops \Rightarrow \langle st_n, dc_n \rangle$, is a successful ADAMS eviction attack, if.f. 1) the admission timeline causes full damage, that is, initial state st_0 contains only benign transactions, ops are adversarial transactions, and the result end state st_n contains only adversarial transactions (in Equation 1), and 2) the attack cost is asymmetrically low, that is, the total adversarial transaction fees in the end state st_n to be charged are smaller, by a multiplicative factor ϵ, than the attack damage measured by the fees of evicted transactions in the initial state st_0 (in Equations 2 and 3). Formally,

$$st_0 \cap st_n = \varnothing \tag{1}$$

$$asym_E(st_0, ops) \stackrel{def}{=} \frac{\sum_{tx \in st_n} tx.fee}{\sum_{tx \in st_0} tx.fee} \tag{2}$$

$$asym_E(st_0, ops) < \epsilon \tag{3}$$

Definition 4.3 (Locking bug oracle): A transaction admission timeline, $\langle st_0 = \varnothing, dc_0 = \varnothing \rangle, ops \Rightarrow \langle st_n, dc_n \rangle$, is a successful ADAMS locking attack, if.f. 1) the admission timeline causes full damage, that is, the mempool is first occupied by adversarial transactions (i.e., st_n contains only adversarial transactions) and then declines the arriving normal transactions (i.e., dc_n contains only normal transactions); the normal transactions and adversarial transactions are sent by different accounts (i.e., Equation 4), and 2) the attack cost is asymmetrically low, that

is, the average adversarial transaction fees in the mempool are smaller, by a multiplicative factor λ, than the attack damage measured by the average fees of victim transactions declined (i.e., Equations 5 and 6). Formally,

$$\cup_{tx\in dc_n} tx.sender \quad \cap \cup_{tx\in st_n} tx.sender = \varnothing \tag{4}$$

$$asym_D(ops) \stackrel{def}{=} \frac{\sum_{tx\in st_n} tx.fee/\|st_n\|}{\sum_{tx\in dc_n} tx.fee/\|dc_n\|} \tag{5}$$

$$asym_D(ops) < \lambda \tag{6}$$

The detailed design rationale of the bug oracles is available in the online supplemental material and is identical to that in the original conference paper [23].

V. Stateful Mempool Fuzzing by MPFUZZ

A. Transaction Symbolization

The key challenge in designing MPFUZZ is the large input space of transaction sequences. Recall that an Ethereum transaction consists of multiple attributes (sender, nonce, *GasPrice*, value, etc.), each defined in a large domain (e.g., 64-bit string). Exploring the raw transaction space is hard; the comprehensive search is inefficient, and randomly trying transactions as done in the state-of-the-art blockchain fuzzers (in § 2) is ineffective.

1) Motivation and Intuition: We propose symbolizing transactions for efficient and effective mempool stateful fuzzing. Our key idea is to map each group of concrete transactions, triggering an equivalent mempool behavior into a distinct symbol so that searching for one transaction is sufficient to cover all other transactions under the same symbol. Transaction symbolization is expected to reduce the search space from possible concrete transactions to the symbol space.

In this work, we manually design seven symbols to represent transactions based on the Ethereum "semantics", that is, how different transactions are admitted by the mempool.

2) Symbol Definitions and Instantiation: Suppose the current state contains adversarial transactions sent from r accounts $A_1, \ldots, A_r$. These accounts have an initial balance of m Ether, where m equals the mempool capacity (say $m = 1000$). The rationale is that these accounts can send at most m adversarial transactions, each minimally spending one Ether, to just occupy a mempool of m slots. A larger value of attacker balance is possible but increases the search space.

Symbol $\mathcal{N}$ defines a transaction subspace covering any normal transactions sent from any benign account and of any nonce, namely $tx\left[\begin{smallmatrix} B* & * \\ * & * \end{smallmatrix}\right]$. In MPFUZZ, Symbol $\mathcal{N}$ is instantiated to concrete transactions of fixed *GasPrice* 3 wei and of value 1 Ether. That is, symbol $\mathcal{N}$ is instantiated to transaction $tx\left[\begin{smallmatrix} B* & 1 \\ * & 3 \end{smallmatrix}\right]$, as shown in Table II. Among all symbols, $\mathcal{N}$ is the only symbol associated with benign sender accounts.

Symbol $\mathcal{F}$ defines the transaction subspace of any future transaction sent from an adversarial account; the future transaction is defined w.r.t. the current state. MPFUZZ instantiates symbol $\mathcal{F}$ to concrete transactions of nonce $m+1$, value 1 Ether, *GasPrice* $m+4$ wei, and any adversarial account $A*$. The instantiating pattern for symbol $\mathcal{F}$ is $tx\left[\begin{smallmatrix} A* & 1 \\ m+1 & m+4 \end{smallmatrix}\right]$.

TABLE II
Symbols, Transactions, and Associated Costs. Tx Refers to the Instantiated Transaction Under a Given Symbol

Symbol	Description	Tx	$cost()$	$opcost()$
$\mathcal{N}$	Benign	$tx\left[\begin{smallmatrix} B* & 1 \\ * & 3 \end{smallmatrix}\right]$	3	3
$\mathcal{F}$	Future	$tx\left[\begin{smallmatrix} A* & 1 \\ m+1 & m+4 \end{smallmatrix}\right]$	0	0
$\mathcal{P}$	Parent	$tx\left[\begin{smallmatrix} A>r & 1 \\ 1 & [4,m+3] \end{smallmatrix}\right]$	$[4, m+3]$	$[4, m+3]$
$\mathcal{C}$	Child	$tx\left[\begin{smallmatrix} A[1,r] & 1 \\ \geq 2 & m+4 \end{smallmatrix}\right]$	$m+4$	1
$\mathcal{O}$	Overdraft	$tx\left[\begin{smallmatrix} A[1,r] & m+1 \\ \geq 2 & m+4 \end{smallmatrix}\right]$	0	0
$\mathcal{L}$	Latent overdraft	$tx\left[\begin{smallmatrix} A[1,r] & m-1 \\ \geq 2 & m+4 \end{smallmatrix}\right]$	0	0
$\mathcal{R}$	Replacement	$tx\left[\begin{smallmatrix} A[1,r] & m-1 \\ 1 & m+4 \end{smallmatrix}\right]$	$m+4$	0

Symbol $\mathcal{P}$ defines the transaction subspace of any parent transaction from an adversarial account w.r.t. the current state. MPFUZZ instantiates $\mathcal{P}$ to a transaction of Pattern $tx\left[\begin{smallmatrix} A\geq r & 1 \\ 1 & [4,m+3] \end{smallmatrix}\right]$. Here, the transaction is fixed with a *GasPrice* in the range $[4, m+3]$ wei, value at 1 Ether, and nonce at 1. The *GasPrice* of a future transaction ($\mathcal{F}$) is $m+4$ wei, which is higher than the price of a parent transaction ($\mathcal{P}$) in $[4, m+3]$ wei. The purpose is to ensure that a future transaction can evict any parent transaction during fuzzing.

Symbol $\mathcal{C}$ defines the transaction subspace of any adversarial child transaction w.r.t. the state. MPFUZZ instantiates $\mathcal{C}$ to a transaction of Pattern $tx\left[\begin{smallmatrix} A[1,r] & 1 \\ \geq 2 & m+4 \end{smallmatrix}\right]$. The instantiated child transaction is fixed at *GasPrice* of $m+4$.

Symbol $\mathcal{O}$ defines the transaction subspace of any adversarial overdraft transaction w.r.t. state st. MPFUZZ instantiates $\mathcal{O}$ to a transaction of Pattern $tx\left[\begin{smallmatrix} A[1,r] & m+1 \\ \geq 2 & m+4 \end{smallmatrix}\right]$. Recall that all adversarial accounts have an initial balance of m Ether.

Symbol $\mathcal{L}$ defines the transaction subspace of any adversarial latent-overdraft transaction w.r.t. state st. MPFUZZ instantiates $\mathcal{L}$ to a transaction of Pattern $tx\left[\begin{smallmatrix} A[1,r] & m-1 \\ \geq 2 & m+4 \end{smallmatrix}\right]$. All adversarial accounts have an initial balance of m Ether.

Symbol $\mathcal{R}$ defines the transaction subspace of any adversarial "replacement" transaction w.r.t., the state. Unlike the symbols above, the transactions under Symbol $\mathcal{R}$ must share the same sender and nonce with an existing transaction in the current state st. MPFUZZ instantiates $\mathcal{R}$ to a transaction of Pattern $tx\left[\begin{smallmatrix} A[1,r] & m-1 \\ 1 & m+4 \end{smallmatrix}\right]$. Here, we simplify the problem and only consider replacing the transaction with nonce 1.

3) Transaction Space Reduction: Given that a concrete transaction in this work consists of four attributes, transactions are defined in a four-dimensional space. We visualize transaction symbols (with an incomplete view) in two two-dimensional spaces in Fig. 2, that is, one of transaction sender and nonce and the other of sender and value. For each symbol, the figures depict the transaction subspace covered by the symbol (in white shapes of black lines) and the transaction pattern instantiated by MPFUZZ under the symbol (in red lines or shapes). In our design, the instantiated transactions are a much *smaller subset* of the defining transaction space. For instance, given state st of transactions sent from account $A_1, \ldots, A_r$, transaction $tx\left[\begin{smallmatrix} A_{r+1} & * \\ 3 & * \end{smallmatrix}\right]$ is within the defining space of symbol $\mathcal{F}$, but MPFUZZ does not instantiate $\mathcal{F}$ by transaction $tx\left[\begin{smallmatrix} A_{r+1} & * \\ 3 & * \end{smallmatrix}\right]$ (but instead to transactions $tx\left[\begin{smallmatrix} A* & 1 \\ m+1 & m+4 \end{smallmatrix}\right]$).

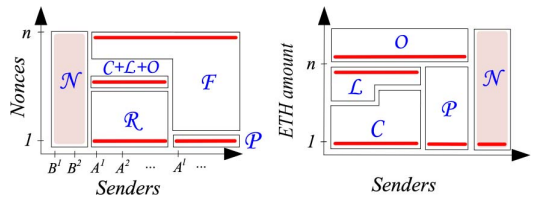

Fig. 2. Symbols and transaction space reduction.

B. Mutation Feedback

We describe how mempool states are symbolized before presenting state-based feedback. In MPFUZZ, a symbolized state st is a list of transaction symbols, ordered first partially as follows: $\mathcal{N} \preceq \mathcal{E} \preceq \mathcal{F} \preceq \{\mathcal{P}, \mathcal{L}, \mathcal{C}\}$. $\mathcal{E}$ refers to an empty slot. State slots of the same symbols are independently instantiated into transactions, except for $\mathcal{P}, \mathcal{C}, \mathcal{L}$. Child transactions (i.e., $\mathcal{C}, \mathcal{L}$) are appended to their parent transaction of the same sender (i.e., $\mathcal{P}$). Across different senders, symbols are ordered by the parent's *GasPrice*.

In MPFUZZ, the feedback of an input ops' is based on the symbolized end state st'. Positive feedback on state st' is determined conjunctively by two metrics: 1) State coverage that indicates state st' is not covered in the corpus, and 2) state promising-ness that indicates how promising the current state is to reach a state satisfying bug oracles.

$$feedback(st', st, sdb) = st\_coverage(st', sdb) == 1 \wedge st\_promising(st', st) == 1 \quad (7)$$

Specifically, state coverage is determined by straightforwardly comparing symbolized state st', as an ordered list of symbols, with all the symbolized states in the corpus. For instance, state $\mathcal{FPCP}$ is different from $\mathcal{FPPC}$. This also implies concrete transaction sequence $tx\left[\begin{smallmatrix} A_1 & * \\ 1 & 4 \end{smallmatrix}\right], tx\left[\begin{smallmatrix} B & * \\ 100 & 100 \end{smallmatrix}\right], tx\left[\begin{smallmatrix} A_2 & * \\ 1 & 5 \end{smallmatrix}\right]$, $tx\left[\begin{smallmatrix} A_1 & * \\ 2 & 10001 \end{smallmatrix}\right]$ is the same with $tx\left[\begin{smallmatrix} A_1 & * \\ 1 & 4 \end{smallmatrix}\right], tx\left[\begin{smallmatrix} B & * \\ 100 & 100 \end{smallmatrix}\right], tx\left[\begin{smallmatrix} A_2 & * \\ 1 & 6 \end{smallmatrix}\right]$, $tx\left[\begin{smallmatrix} A_1 & * \\ 2 & 10 \end{smallmatrix}\right]$, as they are both mapped to the same symbolized state.

How promising a state st' is (i.e., $st\_promising(st')$) is determined as follows: A mutated state st' is more promising than an unmutated state st if any one of the three conditions is met: 1) State st' stores fewer normal transactions (under symbol $\mathcal{N}$) than state st, implying more transactions evicted and higher damage (i.e., $evict\_normal(st', st) = 1$). 2) State st' declines (speculatively) more incoming normal transactions than state st, also implying higher damage (i.e., $decline\_normal(st', st) = 1$). 3) The total fees of adversarial transactions in state st' are lower than those in state st, implying lower attack costs. We consider two forms of costs: concrete cost and symbolized cost. The former, denoted by $cost(st')$, is simply the total fees of adversarial transactions instantiated from a symbolized state st'. The latter, denoted by $opcost(st')$, optimistically estimates the cost of transactions in the current state st' that contributes to a future state triggering test oracle. We will describe the symbolized cost next. Intuitively, $st\_promising(st', st)$ compares the mutated state st' with its parent state st and checks whether the mutation makes progress toward a bug-triggering state. The progress is measured along two dimensions: attack damage and attack cost. The first two predicates, $evict\_normal(st', st)$ and $decline\_normal(st', st)$, capture whether the mutated state causes more damage than the parent state by evicting or declining more normal transactions. The last two predicates, $cost(st') < cost(st)$ and $opcost(st') < opcost(st)$, capture whether the mutated state reduces the adversarial cost, either in the currently instantiated state or in the optimistic symbolized estimate. Therefore, the promisingness metric is computed as a Boolean predicate over these three conditions: a state is considered promising if it improves over its parent state on at least one damage or cost dimension. Formally, state promising-ness is calculated by Equation 8.

$$\begin{aligned} st\_promising(st', st) = {} & evict\_normal(st', st) == 1 \vee \\ & decline\_normal(st', st) == 1 \vee \\ & cost(st') < cost(st) \vee \\ & opcost(st') < opcost(st) \end{aligned} \quad (8)$$

The promisingness metric is re-evaluated after each mutation during fuzzing, and thus evolves together with the explored mempool states. For example, suppose a mutation changes a parent state st into a new state st' that evicts one additional normal transaction while keeping the adversarial cost unchanged. In this case, $evict\_normal(st', st) = 1$, and thus $st\_promising(st', st) = 1$. As another example, suppose a mutation does not increase the number of evicted or declined normal transactions, but reduces the fee of an adversarial transaction needed to maintain the same exploit behavior. Then the state is still considered promising because either $cost(st') < cost(st)$ or $opcost(st') < opcost(st)$ holds. In contrast, if a mutation neither increases damage nor decreases cost, then $st\_promising(st', st) = 0$, and the new state does not receive positive feedback from promisingness. In this way, the metric continuously guides fuzzing toward states that are closer to satisfying the bug oracles with higher damage or lower attack cost.

C. Exploit Extension

The initial short exploit identified on MUT necessitates further automatic development to create a more extensive exploit that is effective within a larger mempool. The key idea in extending the small exploit to a more comprehensive one is to ensure the same admission event observed in the smaller MUT. We first define the admission event as given an arriving transaction tx_i at a mempool of initial state st_0, the admission of tx_i transitions the mempool into an end state st_n. The admission event is denoted by $f(st_0, tx_i) \Rightarrow st_n$. We extract the admission event patterns, representing a state transition from the initial state to the end state. In the pattern, we only symbolize the mempool slots that are updated during the state transition. A pattern is denoted as $\langle st'_0, tx_i, st'_n \rangle$. For example, in the state transition from $\mathcal{NNPC}$ to $\mathcal{NPPC}$ by admitting a transaction $\mathcal{P}_0$, the symbolized state of a mempool slots transitions from $\mathcal{N}$ to $\mathcal{P}$. Thus, we extract the pattern as $\langle \mathcal{N}, \mathcal{P}, \mathcal{P} \rangle$.

Intuitively, an admission event pattern captures the state transition caused by one transaction in the short exploit, abstracting away the unchanged slots in the mempool. Therefore, exploit extension does not require reproducing the entire small-mempool state on the larger mempool. Instead, it aims to replay, on the larger mempool, a sequence of admission behaviors that are consistent with those observed in the short exploit. This makes the extension problem a stateful search over admission event patterns, rather than a direct replay of the short exploit itself.

We extract all the unique admission event patterns in the small exploit and store them into an admission event pattern set (aps). We then construct the attack transaction sequence on the larger-size mempool by aligning the patterns identified from the small exploit with corresponding admission events. This entails validating that the admission event pattern of a given event $f(st_0, tx_i) \Rightarrow st_n$ on the larger-size mempool exists within the aps. Specifically, MPFUZZ first selects ap_i from the set aps whose initial state st_0 matches the state of the large mempool. We then instantiate tx_i of the ap_i and send it to the large mempool. If the state transition of the large mempool $\langle st'_0, tx_i, st'_n \rangle$ aligns with the pattern ap_i, this positive outcome prompts MPFUZZ to proceed with the extension process accordingly. Otherwise, MPFUZZ backtracks by rolling back the mempool state to its state prior to the admission of tx_i and tries the next pattern. MPFUZZ tries the patterns in the decreasing order of $opcost()$, which means if multiple admission event patterns in aps yield positive results, MPFUZZ proceeds the extension process by applying the pattern with the lower $opcost()$.

More concretely, given a current large-mempool state st, MPFUZZ collects from aps all patterns of the form $\langle st'_0, tx_i, st'_n \rangle$ such that st'_0 matches st. We denote the resulting candidate set by $Cand(st)$. A pattern is feasible only if its transaction tx_i can be instantiated and admitted under the current concrete mempool state. For each feasible candidate pattern, MPFUZZ executes the instantiated transaction and observes the resulting large-mempool transition. The pattern is considered matched if the updated slots in the observed transition symbolize to the same initial and end-state pattern as $\langle st'_0, tx_i, st'_n \rangle$. If a match is found, the corresponding transaction is appended to the extended exploit sequence and the new mempool state becomes the starting point for the next extension step. Otherwise, the attempted transaction is discarded, the mempool is rolled back to the previous state, and the next candidate pattern is tried.

The candidate patterns are explored according to increasing attack cost, implemented by prioritizing the pattern with smaller $opcost()$. This ordering favors extension paths that are more likely to preserve the low-cost property of the short exploit. The extension process terminates when the constructed transaction sequence drives the large mempool to a state satisfying the target bug oracle, or when all candidate patterns have been exhausted under backtracking. In this way, exploit extension is a guided search that incrementally reconstructs on the larger mempool the admission behaviors encoded in the short exploit.

Algorithm 1: extendExploit(aps, st)

```
ops_e ← ∅
while oracle(st) = 0 do
    Cand ← matchingPatterns(aps, st)
    sort Cand by increasing opcost()
    matched ← 0
    for each ⟨st'_0, tx_i, st'_n⟩ ∈ Cand do
        if feasible(tx_i, st) then
            ⟨st*, ok⟩ ← send(tx_i, st)
            if ok = 1 and match(st, tx_i, st*, ⟨st'_0, tx_i, st'_n⟩) then
                append tx_i to ops_e
                st ← st*
                matched ← 1
                break
            end if
        end if
    end for
    if matched = 0 then
        return failure
    end if
end while
return ops_e
```

D. State Search Algorithm

We propose a stateful fuzzing algorithm to selectively explore the input space in a way that prioritizes new and promising mempool states toward triggering the ADAMS bug oracle. In particular, MPFUZZ maintains a seed corpus of input-state pairs and selects the next seed according to its energy. During mutation, MPFUZZ appends a previously untried transaction symbol to the current symbolized input, instantiates the mutated input into concrete transactions, and executes them on the target mempool to obtain the resulting end state. The details of the state search algorithm are available in the online supplemental material and are identical to those in the original conference paper [23].

VI. Found Exploits

We have run MPFUZZ across a variety of Ethereum clients, including six leading execution-layer clients on the public-transaction path of Ethereum mainnet (Geth, Besu, Nethermind, Erigon, Reth, and OpenEthereum), three PBS clients (proposer-builder separation) on the mainnet's private-transaction and bundle path (Flashbot builder $v1.11.5$ [16], EigenPhi builder [17] and bloXroute builder-ws [18]), and the clients deployed on three operational Ethereum-like networks (BSC $v1.3.8$ [19] deployed on Binance Smart Chain, go-opera $v1.1.3$ [20] on Fantom, and core-geth $v1.12.19$ [21] on Ethereum Classic).

On the six public-transaction clients, MPFUZZ leads to the discovery of 22 bugs, including 7 conforming to the known DETER attacks on clients of historical versions and 15 new bugs on the clients of the latest versions.

Other clients, including the three PBS clients and three clients on Ethereum-like networks, are mostly forks of the Geth clients, and on them, MPFUZZ discovered 13 bugs of a similar nature to those found on Geth.

The detailed patterns of these bugs are available in the online supplemental material, and are identical to the original conference paper [23].

VII. Evaluation of MPFUZZ

A. Ablation Study

In this section, we describe 4 baseline fuzzers to demonstrate the performance efficacy of the techniques proposed in MPFUZZ as an ablation study.

Baseline B1 - stateless: We implement a stateless fuzzer in the GoFuzz framework. 1) Given a bit string generated by GoFuzz, the fuzzer code parses it into a sequence of transactions; the length of the sequence depends on the length of the bit string. It skips a bitstring that is too short. Specifically, when parsing the bitstring into transaction sequence, every 3 bits are parsed into one transaction, corresponding to 6 adversarial symbolized transactions proposed in MPFUZZ. 2) The fuzzer sends the transaction sequence to the initialized mempool with m normal transactions for execution. 3) It checks if the mempool state after execution meets the bug oracle. 4) It takes code coverage as feedback.

Baseline B2a - non-symbolized state: Compared to MPFUZZ, the only difference is that the baseline B2a takes concrete state in the state coverage, while MPFUZZ takes symbolized state in the state coverage. Specifically, after sending a transaction sequence to the mempool, it sorts the transactions in the mempool by senders and nonces. It then hashes the sorted transactions in the mempool and checks the presence of the hash digest in the seeds. If no hash exists, the transaction sequence increases the concrete-state coverage and is inserted into the corpus. The energy of a state is determined the same way as MPFUZZ, which is determined by its symbolized state.

Baseline B2b - non-symbolized input: We implement the third baseline fuzzer, which is also similar to MPFUZZ but without symbolized input mutation. In each iteration, the baseline fuzzer B2b appends to the current transaction sequence a new transaction. Given the m-slot mempool, the fuzzer tries m values for senders, m values for nonces, m values for GasPrice, and m values for Ether amount. After sending to the mempool the current transaction sequence, it uses the same way to determine the feedback and energy as MPFUZZ.

Baseline B3 - non-promising feedback & energy We build the last baseline fuzzer, which is the same with MPFUZZ except that it removes the state promising-ness from its feedback and the seed selection mechanism based on energy. Specifically, the feedback formula in B3 includes only state coverage ($st\_coverage$) and excludes state promisingness. In addition, B3 uses as the energy the number of invalid transactions on a mempool state. The seed with more invalid transactions in the mempool has higher energy or priority to be selected for the next round of fuzzing.

Experimental settings of fuzzing: We run the four baselines and MPFUZZ against a MUT in a Geth client reconfigured under two settings: A small setting where the mempool is resized to 6 slots $m = 6$ and all fuzzers run for two hours (if test oracle is not triggered), and a medium setting where $m = 16$ and fuzzers run for 16 hours. The fuzzing experiments are run on a local machine with an Intel i7-7700k CPU of 4 cores and 64 GB RAM.

TABLE III
Fuzzing Geth $v1.11.3$'s Mempool (in Minutes) to Detect Exploit XT_3. OT Means Overtime

Settings	B1	B2a	B2b	B3	MPFUZZ
6slot-2 h	OT	0.10	34.61	1.66	0.03
16slot-16 h	OT	0.26	OT	OT	0.06

Results: Table III reports the time used that the first exploit is found under small and medium MUT settings. For the small MUT of 6 slots, baseline B1 cannot find any attack in two hours, B2a, B2b and B3 can find the Exploit XT_3 in 0.10, 34.61 and 1.66 minutes respectively. By contrast, MPFUZZ finds the same exploit (XT_3) in 0.03 minutes. For the medium setting, baselines B1, B2b, and B3 cannot find any exploit in 16 hours, while B2a can find Exploit XT_3 in 0.26 minutes. By contrast, MPFUZZ finds the same exploit in 0.06 minutes.

The above results provide evidence for understanding the impact of different design components on fuzzing performance. Baseline B1 performs the worst because ADAMS exploits are inherently stateful: whether a transaction sequence triggers eviction depends on the evolving mempool state rather than on isolated inputs. As a result, code coverage alone is a weak feedback signal for this problem, since increasing code coverage does not necessarily bring the fuzzer closer to an exploit state. This explains why a stateless design is ineffective in both the small and medium MUT settings.

The poor performance of B2b shows that non-symbolized input mutation suffers from the large concrete transaction space. In B2b, the fuzzer must enumerate combinations of sender, nonce, GasPrice, and Ether amount, which leads to substantial search overhead before reaching a semantically meaningful adversarial transaction. In contrast, MPFUZZ directly mutates over symbolized transaction classes, allowing it to focus on behaviors relevant to bug triggering.

The comparison between B2a and B3 also provides insight into the relative importance of different techniques. B2a is consistently faster than B3, indicating that state promisingness is more helpful than symbolized state coverage in guiding the search. Intuitively, state promisingness provides a stronger exploit-oriented signal by estimating how close an intermediate mempool state is to satisfying the bug oracle, while state coverage mainly encourages exploration diversity. Therefore, although both techniques improve fuzzing, promisingness contributes more directly to exploit discovery efficiency.

Finally, MPFUZZ consistently outperforms all baselines because it combines these three design elements together: symbolized input reduces the search space, symbolized state coverage avoids redundant exploration at the concrete-state level, and state promisingness prioritizes states that are more likely to lead to exploits.

B. Evaluation of Fuzzing Performance

Experimental settings of fuzzing: We evaluate the fuzzing performance of MPFUZZ on four Ethereum clients, Geth $v1.10.11$, Geth $v1.11.4$, Erigon, and Nethermind. We measure two standard fuzzing metrics: the time to first exploit and the

TABLE IV
FUZZING PERFORMANCE MEASURED BY TIME TO FIRST EXPLOIT, WITH EXPLORED STATES AT DISCOVERY SHOWN IN PARENTHESES

Client	Time to 1st exploit (state coverage at discovery)		
	3 slots	16 slots	Default
Geth V1.10.11	1 sec (2)	2 sec (2)	132 min (5122)
Geth V1.11.4	1.92 sec (11)	54 sec (395)	622 min (11264)
Erigon	10 min (10)	10 hours (599)	167 hours (10625)
Nethermind	1 sec (2)	2 sec (2)	7 sec (2)

TABLE V
STATE COVERAGE GROWTH ON MEMPOOL WITH 16 SLOTS

Client	2 min	32 min	128 min	8 hour	16 hour
Geth V1.10.11	749	8451	23439	51532	91428
Geth V1.11.4	372	2496	6482	18602	23286
Erigon	12	70	214	499	1239
Nethermind	15	77	232	579	1433

state coverage growth over time. For time to first exploit, we run MPFUZZ under three MUT settings, including small (3 slots), medium (16 slots), and the clients' default mempool setting. For the state coverage experiment, we report the state coverage growth over time under the 16 -slot MUT setting.

Results: Table IV reports the time used until the first exploit is found, together with the number of explored states at the moment of exploit discovery. Overall, MPFUZZ finds exploits quickly under the reduced MUT settings. For Geth $v1.10.11$, the first exploit is found in 1 second on the 3 -slot MUT and 2 seconds on the 16 -slot MUT, while it takes 132 minutes on the default mempool. For Geth $v1.11.4$, the first exploit is found in 1.92 seconds on the 3 -slot MUT and 54 seconds on the 16 -slot MUT, while it takes 622 minutes on the default mempool. For Nethermind, the first exploit is found in only 1 second on the 3 -slot MUT, 2 seconds on the 16 -slot MUT, and 7 seconds on the default mempool. Erigon is the most challenging target: MPFUZZ needs 10 minutes on the 3 -slot MUT, 10 hours on the 16 -slot MUT, and 167 hours on the default mempool. The reason MPFUZZ is more performant on Geth is that we implemented an external API on Geth to initialize the mempool; in each iteration of fuzzing on Geth, MPFUZZ calls the API to initialize the mempool. In contrast, on Nethermind and Erigon, MPFUZZ restarts the client in each iteration, which is consuming.

Table V reports the state coverage growth over time on a 16 -slot MUT setting. For Geth $v1.10.11$, the explored states increase from 749 at 2 minutes to 91428 at 16 hours. For Geth $v1.11.4$, the state coverage grows from 372 to 23286 over the same period. Similarly, Erigon increases from 12 states to 1239, while Nethermind increases from 15 to 1433.

C. Runtime Cost and Computational Overhead

Experimental settings of runtime evaluation: We evaluate the runtime cost of MPFUZZ by measuring the runtime of the fuzzing phase, the runtime of the exploit extension phase, and the total end-to-end runtime combining both stages. The fuzzing runtime is measured on a 6 -slot MUT, where MPFUZZ first discovers a short exploit. The exploit extension runtime is then measured as the time used to transform the discovered short exploit into an exploit for the larger mempool. The total runtime is the sum of the fuzzing and exploit extension phases.

TABLE VI
RUNTIME COST OF MPFUZZ, INCLUDING FUZZING, EXPLOIT EXTENSION, AND TOTAL RUNTIME

Client	Fuzzing runtime (6-slot MUT)	Exploit extension	Total runtime (fuzzing + extension)
Geth V1.10.11	128 sec	9.3 sec	137.3 sec
Geth V1.11.4	117 sec	9.3 sec	126.3 sec
Erigon	132 min	9.3 sec	132 min 9.3 sec
Nethermind	104 min	9.3 sec	104 min 9.3 sec

Results: Table VI reports the runtime cost of MPFUZZ on four Ethereum clients. For Geth $v1.10.11$ and Geth $v1.11.4$, the fuzzing phase takes 128 and 117 seconds, respectively, while the exploit extension phase takes 9.3 seconds in both cases. The total runtime is 137.3 and 126.3 seconds. For Erigon and Nethermind, the fuzzing phase takes 132 and 104 minutes, respectively, while the exploit extension phase remains lightweight at 9.3 seconds. Their total runtimes are therefore 132 minutes 9.3 seconds and 104 minutes 9.3 seconds.

The result shows that the exploit extension phase incurs only a small runtime overhead compared to fuzzing. In all evaluated clients, the total runtime is dominated by the fuzzing phase. This indicates that the main computational cost of MPFUZZ lies in exploring the mempool state space to discover short exploits, whereas extending a discovered exploit to a larger mempool is efficient in practice.

D. Evaluation of Exploit Extension Algorithm

This section evaluates the effectiveness of the exploit extension algorithm in transforming short exploits found by MPFUZZ on a reduced mempool into valid exploits on a larger actual mempool. Recall that MPFUZZ runs against a small mempool (MUT), and therefore a short exploit found in MUT may not necessarily remain effective after being extended to the actual larger mempool. To evaluate the extension approach, we measure the true and false positives of the extension results. We formally define true and false positives as follows:

Definition 7.1 (TP/FP exploits): Consider a short exploit found by MPFUZZ on a MUT, $\langle st_0, dc_0\rangle, ops \Rightarrow \langle st_n, dc_n\rangle$, and an extended exploit on an actual mempool, $\langle st'_0, dc'_0\rangle, ops' \Rightarrow \langle st'_n, dc'_n\rangle$.

The pair of exploits constitute a true positive eviction attack w.r.t. ϵ (λ), if.f. $st_0 \cap st_n = \varnothing \wedge asym_E(st_0, ops) < \epsilon \wedge\ st'_0 \cap st'_n = \varnothing \wedge asym_E(st'_0, ops') < \epsilon$.

The pair of exploits constitute a false positive eviction attack w.r.t. ϵ (λ), if.f. $st_0 \cap st_n = \varnothing \wedge asym_E(st_0, ops) < \epsilon \wedge (st'_0 \cap st'_n \neq \varnothing \vee asym_E(st'_0, ops') > \epsilon)$.

True/false positives on locking attacks can be similarly defined from bug-oracle definitions 4.2 and 4.3.

Experimental settings: We evaluate the extension algorithm under varying ϵ and λ. First, given a MUT, we run MPFUZZ with varying ϵ (and λ). Increasing ϵ allows MPFUZZ to find more short exploits; for each distinct short exploit, we apply the extension

TABLE VII
TRUE/FALSE POSITIVES (TP/FP) OF EVICTION AND LOCKING ATTACKS DISCOVERED WITH VARYING ϵ AND λ. ✓ MEANS SATISFIED

Exploit	ϵ	$m = 16$ (MUT)		Default m		TP?
		Eq. 1	$asym_E$	Eq. 1	$asym'_E$	
Geth- XT_1	10^{-4}	✓	0	✓	0	✓
Geth- XT_3	.09	✓	0.083	✓	0.0002	✓
Geth- XT_6	.125	✓	0.125	✓	0.0003	✓
Geth- XT_2	0.2	✓	0.167	✓	0.0698	✓
Geth- XT_4	0.23	✓	0.208	✓	0.0768	✓
Geth- XT_5	0.23	✓	0.208	✓	0.0768	✓
Nethermind- XT_1	10^{-4}	✓	0	✓	0	✓
Nethermind- XT_4	0.11	✓	0.104	✓	0.0008	✓
Nethermind- XT_7	0.36	✓	0.355	✓	0.0012	✓
Besu- XT_2	0.2	✓	.167	✓	0.0754	✓
Besu- XT_4	0.23	✓	.208	✓	0.0083	✓
Erigon- XT_4	0.23	✓	0.208	✓	0.0894	✓

Exploit	λ	$m = 16$ (MUT)		Default m		TP?
		Eq. 4	$asym_D$	Eq. 4	$asym'_D$	
Reth- XT_8	0.34	✓	0.34	✓	0.015	✓
OpenEthereum- XT_9	0.46	✓	0.46	✓	0.0439	✓

algorithm to construct an exploit for the default mempool. Second, we evaluate the extended exploits on a default mempool with a local experimental setting. This evaluation allows us to determine whether the extension algorithm preserves exploit validity when moving from the reduced setting to the default mempool.

Results: Table VII presents the true/false positive results of the extended exploits. For instance, when setting ϵ at 0.0001 to find eviction attacks on Geth, MPFUZZ discovers one short exploit XT_1 with $asym_E = 0$ on MUT. After applying the extension algorithm, the resulting exploit remains valid on the actual Geth mempool (of $m = 6144$) with $asym'_E = 0 < 0.0001 = \epsilon$, making it a true positive. Increasing ϵ to 0.23 leads to discovering all six exploits XT_{1-6} on Geth, and all extended exploits remain true positives. The table also reports similar results for other eviction exploits and locking exploits.

Overall, the results show that the extension algorithm is effective in preserving exploit validity under larger mempool sizes. Across the evaluated Ethereum clients, the true-positive rate remains at 100% for $\epsilon \leq 0.36$ and for $\lambda \leq 0.46$. This result supports that the exploit extension algorithm can effectively transform short exploits found in reduced mempools into valid exploits on large actual mempools.

E. Artifact Availability

To support reproducibility, we make MPFUZZ publicly available at https://figshare.com/articles/software/mpfuzz-AE-V2/26068909/6, together with the scripts and configurations used in this study.

VIII. DISCUSSION

A. Generalization to Other Blockchains

While this work focuses on Ethereum and Ethereum-like account-based systems, the core methodology of MPFUZZ is more general. In particular, transaction symbolization, stateful fuzzing, and feedback-guided exploration are not tied to Ethereum alone, but can be applied to other blockchain mempools whose admission logic is driven by transaction semantics and intermediate mempool states. The key observation behind MPFUZZ is that mempool behavior is often determined by a small set of semantically meaningful transaction categories, rather than by all concrete field values. This observation also applies to other blockchain architectures.

For example, in a UTXO-based blockchain such as Bitcoin, admission is driven by UTXO-specific conditions and the current mempool state. One example is a transaction with a missing input, which is invalid and may be handled differently from a transaction whose inputs are all available. Another example is a child transaction whose input comes from an unconfirmed parent transaction, where the mempool must determine whether the parent transaction is already present. A third example is a conflicting transaction that attempts to spend an input already used by another transaction, creating a double-spend conflict. In addition, UTXO systems support fee-bumping behaviors such as replacement or child-pays-for-parent, which also affect admission decisions. Therefore, the general MPFUZZ workflow can be adapted to such systems by redefining the transaction symbols and state abstraction according to the target transaction model and mempool policy.

B. Rationale and Limitations of Transaction Symbolization

The transaction symbols in MPFUZZ are designed to capture the key mempool behaviors relevant to exploit discovery, rather than all concrete transaction values. The rationale is that mempool admission and eviction are typically driven by a small set of semantically meaningful transaction categories, such as whether a transaction is valid, future, replacement, overdraft, or latent-overdraft, and by how these categories interact with the current mempool state. In other words, the attack behaviors depend more on transaction semantics and state transitions than on the exact concrete values of transaction fields. By abstracting concrete transactions into symbols, MPFUZZ reduces the search space while preserving the behaviors most relevant to the bug oracles.

This abstraction is not complete and may introduce over-abstraction. Current symbol design is derived by studying common mempool behaviors in several widely used client implementations and then abstracting the transaction categories that repeatedly trigger those behaviors. However, different clients may contain implementation-specific logic or corner cases that are not reflected in this abstraction. As a result, different concrete transactions mapped to the same symbol may still behave differently under some client implementations, and certain vulnerabilities may depend on concrete field combinations that are not distinguished by the current symbol set. Therefore, like other fuzzing-based approaches, MPFUZZ does not guarantee full coverage of the search space or completeness in vulnerability discovery.

C. Fee Model Assumptions

We make the fee model assumptions explicit. In the testnet evaluation, victim transactions are real transactions sent by normal users on the testnet. In the local evaluation, we use the collected real transaction histories from mainnet as victim transactions. For the attacker, we use the minimal Gas limit of 21000, so the attack cost is determined by *GasPrice* $\times 21{,}000$. The attacker's GasPrice is set relative to victim transactions to trigger the target admission behavior.

To illustrate the sensitivity to victim GasPrice conditions, consider a 16 -slot full mempool. If all victim transactions have GasPrice 100. For the XT_4 attack, the attacker needs to pay the fee for one transaction at a slightly higher GasPrice, say 102. The attack cost is therefore 21000×102, while the victim damage is $21000 \times 100 \times 16$. The attack-cost / victim-damage ratio is thus $102/1600 = 6.375\%$, which is strongly asymmetric. In contrast, if one victim transaction has GasPrice 200 while the other 15 have GasPrice 1. The attacker must set the GasPrice above 200, say 202. The victim damage is $21{,}000 \times (200 + 15 \times 1) = 21{,}000 \times 215$, resulting in a ratio of $202/215 = 93.95\%$. Thus, the attack remains highly asymmetric when victim prices are relatively uniform, but becomes much less asymmetric when a few victim transactions dominate the GasPrice distribution.

D. Responsible Bug Disclosure

We have disclosed the discovered ADAMS vulnerabilities to the Ethereum Foundation, which oversees the bug bounty program across major Ethereum clients, including Geth, Besu, Nethermind, and Erigon. We also reported the found bugs to the developers of Reth, Flashbot, EigenPhi, bloXroute, BSC, Ethereum Classic and Fantom. Besides 7 DETER bugs that MPFUZZ rediscovered, 24 newly discovered ADAMS bugs are reported. As of March 2024, 15 bugs are confirmed: XT_1 (Nethermind, EigenPhi), XT_2 (EigenPhi), XT_3 (EigenPhi), XT_4 (Geth, Erigon, Nethermind, Besu, EigenPhi), XT_5 (Geth), XT_6 (Geth, Flashbot Builder, EigenPhi), XT_7 (Nethermind), and XT_8 (Reth). After our reporting, XT_4/XT_7/XT_8 have been fixed on Geth $v1.11.4$/Nethermind $v1.21.0$/Reth $v0.1.0 - alpha.6$. Bug reporting is documented [31].

E. Ethical Concerns

When evaluating attacks, we only mounted attacks on the testnet and did so with minimal impacts on the tested network. For instance, our attack lasts a short period of time, say no more than 4 blocks produced. Additionally, we obscure the first few digits of the block number in the attack screenshot. In bug reports, we disclose to the developers the mitigation design tradeoff and the risk of fixing one attacks by enabling other attacks. To prevent introducing new bugs, we did not suggest fixes against turning-based eviction and locking.

F. Limitations and Future Works

The exploit generation in this work is not fully automated. Manual tasks include mempool reduction, MPFUZZ setup, etc. Automating these tasks is the future work.

MPFUZZ's bug oracles neither captures complete dependencies among concrete transactions nor guarantees completeness in finding vulnerabilities. Certified mempool security with completeness is also an open problem.

MPFUZZ targets a victim mempool of limited size: the vulnerabilities found in this work are related to transaction eviction, which does not occur in a mempool of infinite capacity. Thus, the MPFUZZ workflow cannot find exploits in the mempool of infinite capacity. It is an open problem whether a mempool of large or infinite capacity has DoS bugs.

IX. Conclusion

This paper presents MPFUZZ, the first mempool fuzzer to find asymmetric DoS bugs by exploring symbolized mempool states and optimistically estimating the promisingness of an intermediate state in reaching bug oracles. Running MPFUZZ on popular Ethereum clients discovers new mempool-DoS bugs, which exhibit various sophisticated patterns, including stealthy mempool eviction and mempool locking.

References

[1] K. Li, Y. Wang, and Y. Tang, "DETER: Denial of ethereum txpool services," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, Virtual Event, Republic of Korea, Nov. 15–19, 2021, Y. Kim, J. Kim, G. Vigna, and E. Shi, Eds., NewYork, NY, USA: ACM, 2021, pp. 1645–1667, doi: 10.1145/3460120.3485369.

[2] K. Baqer, D. Y. Huang, D. McCoy, and N. Weaver, "Stressing out: Bitcoin 'stress testing'," in *Proc. Financial Cryptography Data Secur.-FC 2016 Int. Workshops, BITCOIN, VOTING, and WAHC, Christ Church, Barbados, February 26, 2016, Revised Selected Papers*, in Lecture Notes in Computer Science, J. Clark, S. Meiklejohn, P. Y. A. Ryan, D. S. Wallach, M. Brenner, and K. Rohloff, Eds., vol. 9604, Springer, 2016, pp. 3–18, doi: 10.1007/978-3-662-53357-4.

[3] "Memoria 700 million stuck in 115,000 unconfirmed bitcoin transactions," Accessed: Jul. 1, 2022. [Online]. Available: https://www.ccn.com/700-million-stuck-115000-unconfirmed-bitcoin-transactions/

[4] M. Saad, L. Njilla, C. A. Kamhoua, J. Kim, D. Nyang, and A. Mohaisen, "Mempool optimization for defending against DDoS attacks in pow-based blockchain systems," in *Proc. IEEE Int. Conf. Blockchain Cryptocurrency (ICBC),* Seoul, Korea (South), May 14-17, 2019. IEEE, 2019, pp. 285–292, doi: 10.1109/BLOC.2019.8751476.

[5] "Report: Bitcoin (BTC) mempool shows backlogged transactions, increased fees if so?" Accessed: May 5, 2021. [Online]. Available: https://goo.gl/LsU6Hq

[6] A. Yaish, K. Qin, L. Zhou, A. Zohar, and A. Gervais, "Speculative denial-of-service attacks in ethereum," Cryptology ePrint Archive, Paper 2023/956, 2023. Accessed: Jun. 21, 2026. [Online]. Available: https://eprint.iacr.org/2023/956

[7] Y. Yang, T. Kim, and B. Chun, "Finding consensus bugs in ethereum via multi-transaction differential fuzzing," in *Proc. 15th USENIX Symp. Operating Syst. Des. Implementation (OSDI)*, Jul. 14–16, 2021, A. D. Brown and J. R. Lorch, Eds., USENIX Assoc., 2021, pp. 349–365. [Online]. Available: https://www.usenix.org/conference/osdi21/presentation/yang

[8] F. Ma et al., "LOKI: State-aware fuzzing framework for the implementation of blockchain consensus protocols," in *Proc. 30th Annu. Netw. Distrib. Syst. Secur. Symp., (NDSS)*, San Diego, California, USA, February 27 - March 3, 2023. The Internet Soc., 2023. [Online]. Available: https://www.ndss-symposium.org/ndss-paper/loki-state-aware-fuzzing-framework-for-the-implementation-of-blockchain-consensus-protocols/.

[9] Y. Chen, F. Ma, Y. Zhou, Y. Jiang, T. Chen, and J. Sun, "Tyr: Finding consensus failure bugs in blockchain system with behaviour divergent model," in *Proc. IEEE Symp. Secur. Privacy (SP)*, Los Alamitos, CA, USA: IEEE Computer Soc., May 2023, pp. 2517–2532, doi: 10.1109/SP46215.2023.10179386.

[10] "Geth: the go client for ethereum," Accessed: Jun. 21, 2026. [Online]. Available: https://www.ethereum.org/cli\#geth

[11] "Nethermind ethereum client," Accessed: Jun. 21, 2026. [Online]. Available: https://nethermind.io/
[12] "Erigon," Accessed: Jun. 21, 2026. [Online]. Available: https://github.com/ledgerwatch/erigon
[13] "Hyperledger Besu," Accessed: Jun. 21, 2026. [Online]. Available: https://www.hyperledger.org/use/besu
[14] "Reth: Modular, contributor-friendly and blazing-fast implementation of the ethereum protocol," Accessed: Jun. 21, 2026. [Online]. Available: https://github.com/paradigmxyz/reth
[15] "Parity ethereum is now openethereum: Fast and feature-rich multi-network ethereum client," Accessed: Jun. 21, 2026. [Online]. Available: https://openethereum.github.io/
[16] "Flashbots mev-boost block builder," Accessed: Jun. 21, 2026. [Online]. Available: https://github.com/flashbots/builder
[17] "Eigenphi builder," Accessed: Jun. 21, 2026. [Online]. Available: https://github.com/eigenphi/builder
[18] "bloxroute builder," Accessed: Jun. 21, 2026. [Online]. Available: https://github.com/bloXroute-Labs/builder-ws
[19] "BSC client," Accessed: Jun. 21, 2026. [Online]. Available: https://github.com/bnb-chain/bsc
[20] "go-opera," Accessed: Jun. 21, 2026. [Online]. Available: https://github.com/Fantom-foundation/go-opera/tree/master
[21] "core-geth," Accessed: Jun. 21, 2026. [Online]. Available: https://github.com/etclabscore/core-geth
[22] "Go fuzzing," Accessed: May 31, 2023. [Online]. Available: https://go.dev/security/fuzz/
[23] Y. Wang, Y. Tang, K. Li, W. Ding, and Z. Yang, "Understanding ethereum mempool security under asymmetric DoS by symbolized stateful fuzzing," in *Proc. 33rd USENIX Secur. Symp. (USENIX Secur)*, Philadelphia, PA: USENIX Assoc., Aug. 2024. pp. 4747–4764. [Online]. Available: https://www.usenix.org/conference/usenixsecurity24/presentation/wang-yibo
[24] Y. Zhao, X. Wang, L. Zhao, Y. Cheng, and H. Yin, "Alphuzz: Monte Carlo search on seed-mutation tree for coverage-guided fuzzing," in *Proc. 38th Annu. Comput. Secur. Appl. Conf., (ACSAC)*, New York, NY, USA: ACM, 2022, pp. 534–547, doi: 10.1145/3564625.3564660.
[25] K. Böttinger, P. Godefroid, and R. Singh, "Deep reinforcement fuzzing," 2018, *arXiv:1801.04589*.
[26] D. Wang, Z. Zhang, H. Zhang, Z. Qian, S. V. Krishnamurthy, and N. Abu-Ghazaleh, "SyzVegas: Beating kernel fuzzing odds with reinforcement learning," in *Proc. 30th USENIX Secur. Symp. (USENIX Secur),* USENIX Assoc., Aug. 2021, pp. 2741–2758. [Online]. Available: https://www.usenix.org/conference/usenixsecurity21/presentation/wang-daimeng
[27] C. Aschermann, S. Schumilo, A. Abbasi, and T. Holz, "Ijon: Exploring deep state spaces via fuzzing," in *Proc. IEEE Symp. Secur. Privacy (SP),* 2020, pp. 1597–1612.
[28] Y. Chen, F. Ma, Y. Zhou, M. Gu, Q. Liao, and Y. Jiang, "Chronos: Finding timeout bugs in practical distributed systems by deep-priority fuzzing with transient delay," in *Proc. IEEE Symp. Secur. Privacy (SP),* 2024, pp. 1939–1955.
[29] T. Lyu et al., "Monarch: A fuzzing framework for distributed file systems," in *Proc. USENIX Annu. Tech. Conf. (USENIX ATC)*, Santa Clara, CA: USENIX Assoc., Jul. 2024, pp. 529–543, [Online]. Available: https://www.usenix.org/conference/atc24/presentation/lyu
[30] J. Krupp, I. Grishchenko, and C. Rossow, "AMPFUZZ: Fuzzing for amplification DDoS vulnerabilities," in *Proc. 31st USENIX Secur. Symp. (USENIX Security)*, Boston, MA: USENIX Assoc., Aug. 2022, pp. 1043–1060. [Online]. Available: https://www.usenix.org/conference/usenixsecurity22/presentation/krupp
[31] "Flashbots block builder," Accessed: Jun. 21, 2026. [Online]. Available: https://sites.google.com/view/mpfuzz